MATH 463 Final Project

Singular Value Decomposition and Image Compression

Owen Barnes, Raphael Zeldin, Isabel Cole, Efosa Owie, Eric Filipkowski

May 2026

1 Introduction

As society becomes increasingly digital, the demand for data storage continues to grow despite finite storage capacity. One solution is to reduce storage requirements through data compression, allowing more information to be stored in smaller file sizes. The principal goal of compression is to remove redundancy by retaining only the most significant information required to reconstruct the original data.

Singular Value Decomposition (SVD) compresses data by decomposing a matrix into orthogonal components and retaining only the largest singular values, which capture the majority of the information contained in the original data. The effectiveness of SVD for compression arises from the fact that many images can be well-approximated by low-rank matrices. By retaining only the largest singular values and their corresponding singular vectors, it is possible to construct a lower-rank approximation that preserves the dominant structural features of the image while significantly reducing storage requirements.

SVD is one of many image compression techniques that exist. A popular technique used in JPEG images is the *Discrete Cosine Transform* (DCT) method. This method is generally preferred as it tends to have a higher compression ratio than SVD and a lower Mean Square Error [1]. We seek to study SVD because it has the capability to compress and reduce storage of images [2] as a technique applied to a matrix image. Though SVD may not be a competitive image compression technique on its own, variations of SVD that include a type of pre-processing step have been shown to perform better than DCT techniques [3]. Other areas that SVD performs well in include digital image watermarking where SVD provides strong resistance against noised-based copyright infringement attacks [4]. We will examine SVD for image compression to better understand how linear algebra can be applied to the real-world issue of limited data storage.

In this paper, we will analyze how the number of retained singular values affects both compression efficiency and image reconstruction quality. Numerical experiments will be demonstrated to evaluate the performance of SVD-based compression.

2 Background

2.1 Singular Value Decomposition

Throughout this paper, we will denote vectors in boldface as $\mathbf{v}$ and the Hermitian (conjugate transpose) as $\mathbf{v}^H$. Matrices will be represented as capital letters such as A.

Consider the general case, in which A may contain real or complex entries. Beginning with a matrix $A \in \mathbb{C}^{m\times n}$, Singular Value Decomposition allows us to write A as $A = U\Sigma V^H$, where

- $U \in \mathbb{C}^{m\times m}$, is unitary, and its columns are eigenvectors of AA^H;
- $V \in \mathbb{C}^{n\times n}$ is unitary, and its columns are eigenvectors of $A^H A$;

- $\Sigma \in \mathbb{C}^{m\times n}$ is diagonal, with non-negative singular values σ_i on the diagonal. These singular values are the square roots of the nonzero eigenvalues of AA^H and A^HA.

For real-valued matrices, $A^H = A^T$, so the decomposition simplifies to

$$A = U\Sigma V^T.$$

Here $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal matrices, satisfying

$$U^TU = I, \qquad V^TV = I.$$

To facilitate the construction of the rank-k approximation matrix A_k, we adopt the ordering convention

$$\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_r \geq 0, \qquad r = \text{rank}(A),$$

where singular values are ordered by their relative magnitude.

2.2 SVD for Image Compression

Singular value decomposition is often employed as a technique for image compression, reducing data storage requirements while maintaining high visual fidelity. A grayscale image can be represented as a real-valued matrix $A \in \mathbb{R}^{m\times n}$, with entries corresponding to pixel intensity. A colored image can similarly be represented as a collection of separate real-valued matrices $\{A_R, A_G, A_B\}$, one for each color channel (e.g., red, green, and blue). Some image formats also include an additional *alpha* channel matrix A_α for transparency [5]. In practice, color images can be converted to grayscale by combining or discarding color channels, allowing the image to be represented as a single matrix A.

By retaining only the largest k *singular values*, σ_i, and their associated *singular vectors*, $\mathbf{u}_i$ and $\mathbf{v}_i$, the approximation preserves the most significant visual features of the image while discarding less significant components. Since singular values are ordered in decreasing magnitude, retaining the first k terms results in a reduced storage requirement with minimal loss in visual fidelity [5].

The rank-k approximation can be written as

$$A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T, \qquad \text{or in matrix form,} \qquad A_k = U_k S_k V_k^T.$$

Mathematically, the rank-k approximation for images can be constructed as follows [6]:

1. Compute the singular value decomposition $A = U\Sigma V^T$, where U and V are orthogonal matrices of size $m \times m$ and $n \times n$, respectively, and Σ is an $m \times n$ diagonal matrix with nonnegative singular values in decreasing order;
2. Form U_k by taking the first k columns of U $(m \times k)$;
3. Form V_k^T by taking the first k rows of V^T $(k \times n)$;
4. Form Σ_k by retaining the k largest singular values along the diagonal of Σ;
5. Construct the approximation $A_k = U_k\Sigma_k V_k^T$.

The quality of this approximation is measured using the Frobenius Norm Error:

$$\|A - A_k\|_F = \sqrt{\sum_{i,j} (A_{i,j} - (A_k)_{i,j})^2}.$$

The resulting rank-k matrix A_k, requires $k(m+n+1)$ of storage compared to mn for the original matrix A. This reduction is particularly advantageous for compressing high-resolution images when $k \ll \min(m,n)$. A standard approach for choosing k is to choose such that the sum of the top k singular values is at least some constant c times as big as the sum of other singular values [6].

The Eckart-Young-Mirsky theorem provides the theoretical foundation for low-rank approximation using Singular Value Decomposition [7, pp. 79–80]. Let $A \in \mathbb{C}^{m\times n}$ have the SVD

$$A = \sum_{i=1}^{r} \sigma_i \mathbf{u_i} \mathbf{v_i^H},$$

where $\sigma_1 \geq \sigma_2 \geq \cdots \sigma_r \geq 0$, and $r = \text{rank}(A)$. Then the rank-k truncated SVD is defined by

$$A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^H.$$

Then A_k is a best approximation to A among all matrices of rank at most k. That is, for any matrix B with $\text{rank}(B) \leq k$ and any unitarily invariant norm $\|\cdot\|$, such as the Frobenius and the spectral norms [8],

$$\|A - A_k\| \leq \|A - B\|.$$

For the Frobenius norm, denoted by $\|\cdot\|_F$ and the spectral norm, denoted by $\|\cdot\|_2$, the approximation error can be expressed explicitly in terms of the singular values [9, p. 6-10]:

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2}, \qquad \|A - A_k\|_2 = \sigma_{k+1}.$$

It follows that the error depends entirely on those singular values which are discarded in the truncation, namely $\sigma_{k+1}, \ldots, \sigma_r$.

Because $\|A\|_F^2 = \sum_{i=1}^{r} \sigma_i^2$, the squared Frobenius norm reflects the total energy of the matrix [10, p. 39]. Thus, the Frobenius norm error $\|A - A_k\|_F$ corresponds to the energy not captured by the first k singular components. In the context of image compression, this theorem ensures that the truncated matrix A_k produces the best possible rank-k approximation of the image matrix [11, p. 90–93].

Since the singular values σ_i are ordered in decreasing magnitude, the effectiveness of Singular Value Decomposition depends on the rate at which they decay. If the singular values decay rapidly, a small value of k captures most of the matrix's energy, yielding a high-quality approximation with lower storage cost. Conversely, if the singular values decay slowly, a larger value of k is required to achieve the same level of accuracy, resulting in increased storage requirements[10, p. 311].

3 Methodology

3.1 Matrix Normalization

We developed the Python program `imagecompression.py` to perform SVD-based image compression analysis. As the first step in the workflow, *matrix normalization* converts the input image (`.jpg`, `.png`, and other formats supported by `skimage`) [12] into a grayscale floating-point `NumPy` array suitable for further analysis [13].

Using the `img_as_float()` method from `skimage`, pixel values are scaled from integer ranges (0–255) to floating-point values in $[0, 1]$. If the image is in the RGB (colored) format, it is converted to grayscale

(black and white). Any alpha (transparency) channel is removed in this process so that the image can be represented as a 2-dimensional `NumPy` array $A \in \mathbb{R}^{m \times n}$.

When the `print` flag is set to `True`, the code displays a preview of both the original and the converted grayscale image to verify that the transformation was performed correctly.

For our analysis, we chose an example image of colorful balloons to illustrate the process. The resulting color to grayscale conversion is shown in Figure 1. Additional images will be included with the code submission.

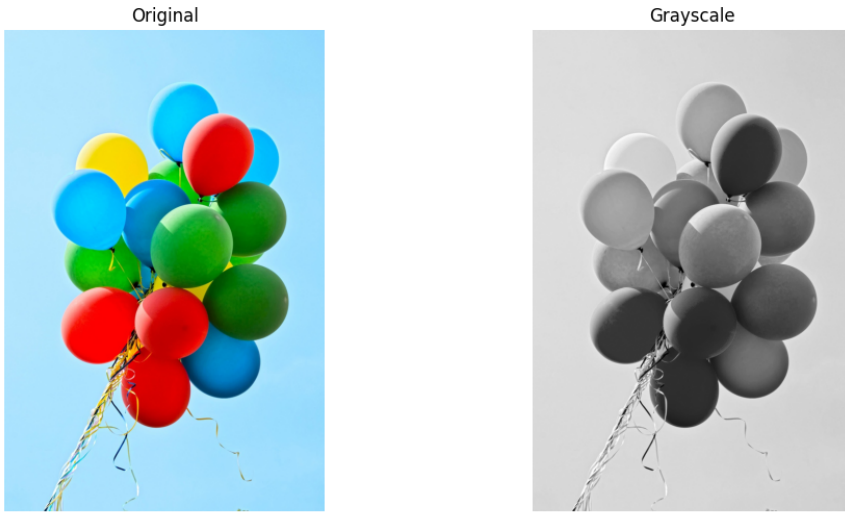


Figure 1: Original color image and its grayscale representation after normalization.

3.2 SVD Computation

The matrix A is decomposed using `numpy.linalg.svd` into the components U, Σ, and V^H, where $A = U\Sigma V^H$. The singular values in Σ are ordered in decreasing magnitude and quantify the relative importance of the corresponding singular vector components. By retaining only the first k singular values and their associated singular vectors, the code constructs the rank-k approximation

$$A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^H.$$

In practice, the *compact form* of the SVD was computed using `numpy.linalg.svd` with the argument `full_matrices=False`. For a matrix $A \in \mathbb{R}^{m \times n}$ with rank r, the compact SVD is given by

$$A = U_r \Sigma_r V_r^H,$$

where $U_r \in \mathbb{R}^{m \times r}$, $\Sigma_r \in \mathbb{R}^{r \times r}$, and $V_r \in \mathbb{R}^{n \times r}$. The compact SVD removes redundant components associated with zero singular values [14]. The compact form of the SVD preserves the full nonzero singular structure of the matrix while reducing storage requirements and computational complexity.

3.3 Orthogonality Verification

Using `NumPy`'s Singular Value Decomposition routine (`numpy.linalg.svd`), we computed the compact form of Singular Value Decomposition, obtaining the arrays U, S and V^H. Here, U and V^H are unitary matrices, $V^H = \overline{V}^T$, and S is the vector of singular values σ_i.

While `NumPy` defines the matrices U and V^H as unitary, in this case they contain only real-valued entries. Therefore, they are in fact orthogonal matrices. Orthogonal matrices are real-valued square matrices whose columns (and rows) form an orthonormal set of vectors. That is, for any pair of column (or row) vectors, $\mathbf{q}$, in an orthogonal matrix Q, we have

$$\begin{aligned} \mathbf{q}_i^T \mathbf{q}_j &= 0, \quad \text{if} \quad i \neq j, \\ \mathbf{q}_i^T \mathbf{q}_j &= 1, \quad \text{if} \quad i = j. \end{aligned}$$

Hence, for an orthogonal matrix Q of size n,

$$Q^T Q = I_n.$$

To verify the orthogonality of matrices V and U, we computed the matrix products $U^T U$ and $V^H (V^H)^T$. In the compact SVD, $U \in \mathbb{R}^{m \times r}$ and $V^H \in \mathbb{R}^{r \times n}$, so both products are matrices of size $r \times r$. Therefore, if the columns are orthonormal, we expect

$$U^T U = I_r, \qquad \text{and} \qquad V^H (V^H)^T = V^T V = I_r.$$

To ensure against any dimensional compatibility issues, we coded this as two separate identity matrices I_u and I_v which are constructed to the same dimensions of the products $U^T U$ and $V^T V$ using `numpy.eye`.

We compared these products to the identity matrices I_u and I_v using `numpy.allclose`, which accounts for floating-point round-off error. Additionally, we computed the Frobenius Norm of the difference between each product and the identity matrix using `numpy.linalg.norm`, that is, $\|U^T U - I_u\|_F$ and $\|V^T V - I_v\|_F$.

The Frobenius Norm is given by

$$\|A\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} |a_{ij}|^2}, \qquad \text{or equivalently,} \qquad \|A\|_F = \sqrt{\text{trace}(A^T A)}.$$

If the matrices are orthogonal, we expect

$$\|U^T U - I_u\|_F = 0 \qquad \text{and} \qquad \|V^T V - I_v\|_F = 0.$$

Numerically, exact zeros are near impossible due to floating-point round-off error. In our experiments, the residual norms were extremely small, $\mathcal{O}(10^{-14})$, indicating that the matrices are orthogonal within numerical precision.

Item	Value
$\|U^T U - I_u\|$	7.53×10^{-14}
$\|V^T V - I_v\|$	7.98×10^{-14}
U numerically orthogonal	True
V numerically orthogonal	True

Table 1: Numerical verification of the orthogonality conditions for U and V. Orthogonality is confirmed by the near-zero values of $\|U^T U - I\|$ and $\|V^T V - I\|$.

4 Visualization of the SVD

Singular Value Decomposition can be interpreted geometrically as a sequence of linear transformations. In this section, we will visualize this interpretation by showing the action of the SVD on a test vector, $\bar{s}$, and decompose the resulting transformation into three constituent steps. Visualization was performed using `matplotlib` [15].

4.1 Geometric Interpretation

Given the Singular Value Decomposition $A = U\Sigma V^H$, the matrix transformation can be interpreted geometrically as a sequence of three intermediate transformations:

1. $V^H\bar{s}$, a rotation by V^H in the input space,
2. $\Sigma V^H\bar{s}$, a scaling by Σ, which stretches or compresses along the principal axes,
3. $U\Sigma V^H\bar{s}$, a rotation by U in the output space.

We expected the second transformation to rescale the vector without changing its direction. However, because the singular values scale each coordinate direction by different amounts according to their magnitudes, the transformation may also alter the direction of the vector relative to its original orientation, as shown in the middle panel of Figure 2. This phenomenon is known as *anisotropic scaling*, where scaling occurs non-uniformly across different directions [16]. Consequently, the transformed vector was observed to change direction in addition to changing magnitude. The observed angular change was verified using the original high-dimensional vectors prior to projection into the two-dimensional visualization basis; therefore, the effect is not an artifact of the visualization.

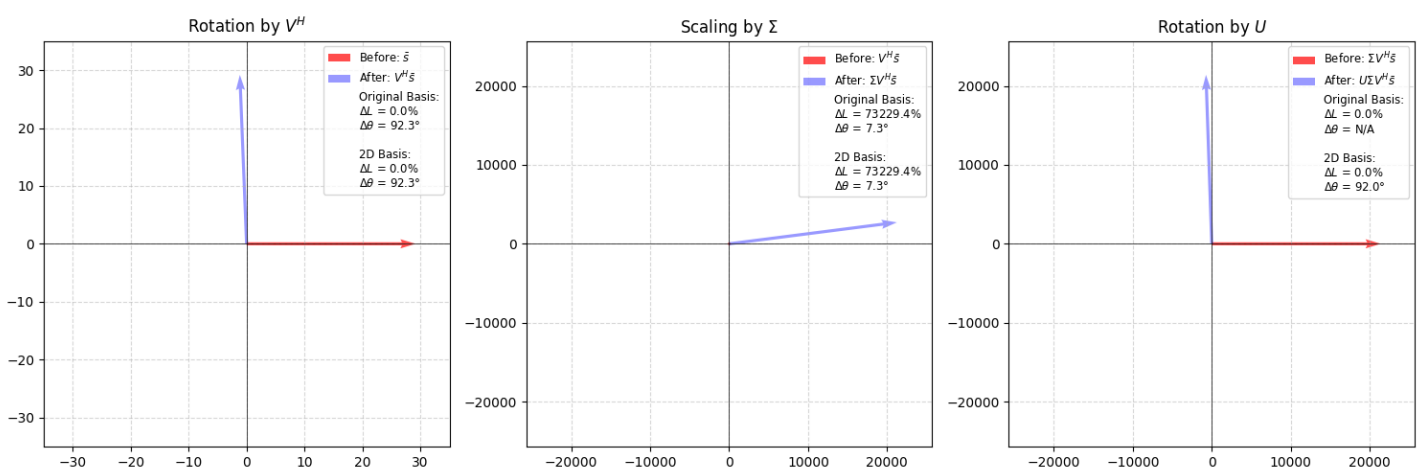


Figure 2: Visualization of the matrix operations applied to the test vector $\bar{s}$. In each stage, the red vector represents the input and is aligned with the x-axis, while the blue vector represents the transformed output. Each panel is displayed in a different two-dimensional basis constructed for visualization purposes. The "Original Basis" quantities shown in the legend correspond to the true angle and length changes in the original vector space prior to projection into the two-dimensional visualization basis.

4.2 Implementation of Visualization

We tracked the transformation of a test vector $\bar{\mathbf{s}}$ through each stage of the SVD. Constructing a two-dimensional visualization of the SVD transformations presented a challenge because the vectors involved lie in different high-dimensional vector spaces. To visualize each transformation, we projected the input and output vectors onto a two-dimensional subspace spanned by those vectors. Each transformation is therefore displayed in its own local two-dimensional coordinate system. To construct these coordinate systems, we used the *Gram–Schmidt* process to build an orthonormal basis from the span of the input and output vectors at each stage. When the vectors are linearly independent, this produces an orthonormal basis for their span. If the vectors are collinear, an auxiliary orthogonal direction is instead constructed using standard basis vectors. One limitation of using separate bases for each plot is that angles and lengths are not visually comparable across subplots. Because each subplot uses a different orthonormal basis, the coordinate systems are local to each transformation. Each subplot should be interpreted as depicting only the local transformation on the input vector (red, aligned with the x-axis) to the resulting output vector (blue).

For our test vector, we used $\bar{\mathbf{s}} \in \mathbb{R}^n$, a `NumPy` vector of ones of length n. Specifically, we computed $V^H\bar{s}$, $\Sigma V^H\bar{s}$, and $U\Sigma V^H\bar{s}$, and visualized the change from the input vector to the output vector at each stage of the transformation. The visualization reports both the original geometric transformation (scaling and rotation) and the projected 2D transformation for each stage of the SVD process.

The rotation angle θ was calculated for both the original vectors and the 2-D visualization vectors [17] separately by using the formula

$$\cos\theta = \frac{\mathbf{v}_2 \cdot \mathbf{v}_1}{\|\mathbf{v}_2\|\,\|\mathbf{v}_1\|}, \qquad \theta = \arccos(\cos\theta),$$

Since the compact SVD of $A \in \mathbb{R}^{m\times n}$ is given by $A = U\Sigma V^H$, where r denotes the rank of A, and

$$U \in \mathbb{R}^{m\times r}, \qquad \Sigma \in \mathbb{R}^{r\times r}, \qquad V^H \in \mathbb{R}^{r\times n},$$

the Panel-3 input vector $\mathbf{v}_2 := \Sigma V^H\bar{s}$ lies in $\mathbb{R}^r$, while the output vector $\mathbf{v}_3 := U\Sigma V^H\bar{s}$ lies in $\mathbb{R}^m$. Because these vectors belong to subspaces of different dimensions, r and m, a direct dot-product angle comparison between initial $\mathbf{v_2}$ and final $\mathbf{v_3}$ is not defined for $r \neq m$. Consequently, the final panel reports only the rotation of the projected vectors within the local 2-dimensional basis.

5 Compression and Analysis

5.1 Spectral Analysis

The k-value represents the number of retained singular values used in reconstructing the image. Lower values of k generally produce lower-quality reconstructions, while larger values preserve more information from the original image. As shown in Figure 3, the logarithmic-scale plot of the singular values exhibits a rapid initial decay followed by a slower decline. This behavior indicates that most of the image energy is concentrated in the leading singular values, while the remaining singular values contribute relatively little detail. Consequently, accurate low-rank approximations can be achieved using relatively small values of k, enabling effective image compression with minimal loss in visual fidelity.

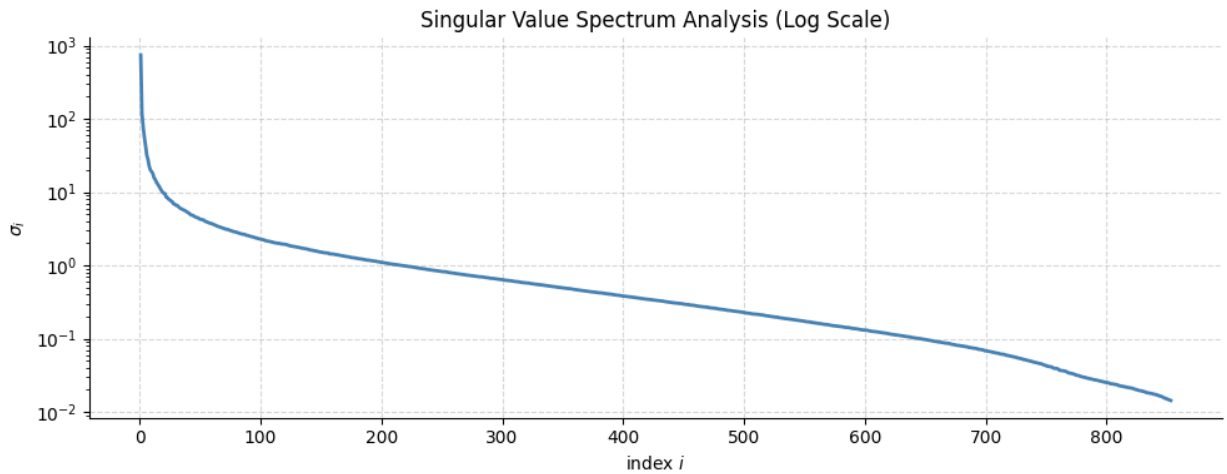


Figure 3: Singular value spectrum of the image matrix on a logarithmic scale, illustrating the rapid decay of σ_i and the effectiveness of low-rank approximations.

5.2 Error Quantification

We can calculate the reconstruction error between our original image matrix A and its rank-k approximation A_k using the *relative* Frobenius norm error, given by

$$\text{Error} = \frac{||A - A_k||_F}{||A||_F}.$$

Since the squared Frobenius norm corresponds to the total energy contained in the singular values of the matrix, we can also measure how much information is retained in the compressed image using the energy ratio

$$\text{Energy}(k) = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{n} \sigma_i^2}$$

As the value of k increases, the relative error decreases, indicating that higher-rank approximations retain more information from the original image. Additionally, the rate of decrease in the relative error diminishes as k becomes larger, suggesting diminishing returns in reconstruction accuracy for sufficiently large values of k. Figure 4 depicts the relative error for several rank-k approximations.

5.3 Optimizing the k-value

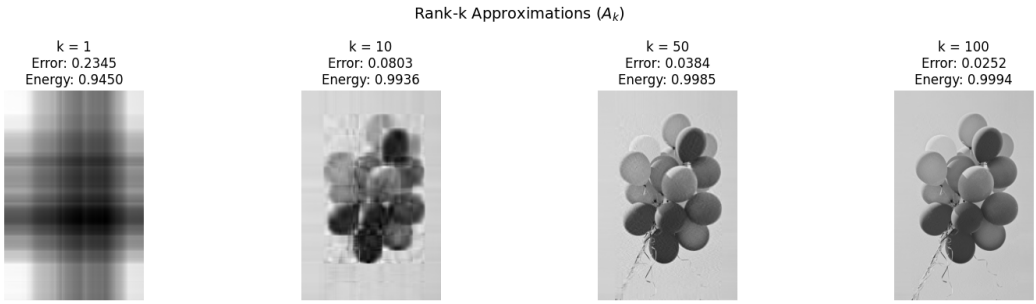
When trying to find the optimal k-value of A, we use the 2-norm of the matrix:

$$||A||_2 = \sqrt{\lambda_{\max}(A^T A)} = \sigma_1,$$

where σ_1 is the largest singular value of A. Equivalently, the 2-norm is the largest entry on the diagonal of Σ. Using a prescribed tolerance $\varepsilon = 0.001$, we retain the singular values satisfying $\sigma_i \geq \varepsilon \cdot \sigma_1$. By the Eckart–Young-Mirsky theorem,

$$||A - A_k||_2 = \sigma_{k+1}$$

By choosing a choosing k such that $\sigma_{k+1} < \varepsilon \cdot \sigma_1$, one can guarnatee that the approximation error is small relative to the size of A. Applying this criterion to our example yields the optimal truncation value $k_{\text{opt}} = 269$.



k	Relative Error	Energy
1	2.3450×10^{-1}	9.4501×10^{-1}
10	8.0296×10^{-2}	9.9355×10^{-1}
50	3.8424×10^{-2}	9.9852×10^{-1}
100	2.5202×10^{-2}	9.9936×10^{-1}

Figure 4: Low-rank approximations A_k of the image matrix for selected values of k. As k increases, the relative reconstruction error decreases while the retained energy approaches 1.

5.4 Compression Ratio

For a given k-value, we can compute the compression ratio, which measures the storage reduction achieved by the compressed image. The compression ratio is given by

$$\text{Compression Ratio} = \frac{m \times n}{k(m+n+1)},$$

where mn represents the storage required for the original matrix and $k(m+n+1)$ represents the storage required for the rank-k approximation. As shown in Figure 5, the compression ratio of the rank-k approximation can be compared directly with that of the original image matrix A. For $k_{\text{opt}} = 269$, the reconstruction preserves nearly all of the spectral energy while maintaining a low relative reconstruction error. Since the compression ratio exceeds 1, the truncated SVD representation achieves storage reduction relative to the original image matrix.

Reconstruction	Relative Error	Energy	Compression Ratio
$k = 269$	9.5884×10^{-3}	9.9991×10^{-1}	1.9020
Full Rank ($k = 853$)	4.8290×10^{-15}	1.0000	1.0000

Figure 5: Comparison of the optimal rank-k approximation and the full-rank reconstruction.

6 Conclusion

Singular Value Decomposition (SVD) is an effective technique for image compression because it reduces the amount of data required to represent an image while preserving most of its essential visual information. In the

SVD representation of an image matrix, the singular values are ordered according to their contribution to the overall structure of the image. By retaining only the most significant singular values and their corresponding singular vectors, a lower-rank approximation of an image can be constructed that maintains much of the visual quality of the original image. With an appropriate rank selection, substantial reductions in storage requirements can often be achieved while preserving a high level of visual quality.

References

[1] U. Bhade, S. Kumar, P. Dwivedy, S. Soofi, and A. Ray, "Comparative study of dwt, dct, btc and svd techniques for image compression," in *2017 International Conference on Recent Innovations in Signal processing and Embedded Systems (RISE)*, 2017, pp. 279–283.

[2] H. Prasantha, H. Shashidhara, and K. Balasubramanya Murthy, "Image compression using svd," in *International Conference on Computational Intelligence and Multimedia Applications (ICCIMA 2007)*, vol. 3, 2007, pp. 143–145.

[3] A. Ranade, S. S. Mahabalarao, and S. Kale, "A variation on svd based image compression," *Image and Vision Computing*, vol. 25, no. 6, pp. 771–777, 2007. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0262885606002083

[4] J. Varghese, O. B. Hussain, S. Subash, and A. Razak, "An effective digital image watermarking scheme incorporating dct, dft and svd transformations," *PeerJ Computer Science*, vol. 9, p. e1400, 2023. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10403172/

[5] J. Jiang, "Image compression using singular value decomposition," 2025. [Online]. Available: https://arxiv.org/abs/2512.16226

[6] T. Roughgarden and G. Valiant, "Lecture notes: The singular value decomposition (svd) and low-rank matrix approximations," 2017.

[7] Z. Zhang, "The singular value decomposition, applications and beyond," *arXiv preprint arXiv:1510.08532*, 2015. [Online]. Available: https://arxiv.org/abs/1510.08532

[8] N. J. Higham, "What is a unitarily invariant norm?" 2021, accessed: 2026-05-05. [Online]. Available: https://nhigham.com/2021/02/02/what-is-a-unitarily-invariant-norm/

[9] J. Hopcroft and R. Kannan, "Singular value decomposition," 2012, chapter 4 of the course notes "Computer Science Theory for the Information Age", Carnegie Mellon University. [Online]. Available: https://www.cs.cmu.edu/~venkatg/teaching/CStheory-infoage/book-chapter-4.pdf

[10] C. C. Aggarwal, *Linear Algebra and Optimization for Machine Learning: A Textbook*. Springer, 2020.

[11] M. Embree, "Matrix methods for computational modeling and data analytics," 2019, virginia Tech, Spring 2019 lecture notes. [Online]. Available: https://personal.math.vt.edu/embree/cmda3606/chapter6.pdf

[12] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, T. Yu, and the scikit-image contributors, "scikit-image: Image processing in python," *PeerJ*, vol. 2, p. e453, 2014. [Online]. Available: https://doi.org/10.7717/peerj.453

[13] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[14] R. D. Wilkinson, "Singular value decomposition," 2024, accessed: 2026-05-05. [Online]. Available: https://rich-d-wilkinson.github.io/MATH3030/3-3-linalg-SVD.html

[15] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[16] Wikipedia, "Scaling (geometry)," 2025, accessed: 2026-05-09. [Online]. Available: https://en.wikipedia.org/wiki/Scaling_(geometry)

[17] Wikipedia, "Vector algebra relations," 2025, accessed: 2026-05-07. [Online]. Available: https://en.wikipedia.org/wiki/Vector_algebra_relations

7 Group Member Contributions

1. **Owen Barnes**
   (a) `imagecompression.py`
      i. `part4_5_grapher()`
      ii. `spectral_analysis_and_error_quantification()`
      iii. `compression_ratio()`
   (b) **Methodology**
      i. Compression and Analysis
      ii. Spectral Analysis
      iii. Error Quantification
      iv. Optimizing the k-value
      v. Compression Ratio
2. **Raphael Zeldin**
   (a) `imagecompression.py`
      i. `svdform()`
      ii. `matrix_normalization()`
      iii. `orthogonality_check()`
      iv. `visualize_svd()`
   (b) **Background**
      i. SVD for Image Compression
   (c) **Methodology**
      i. SVD Computation
      ii. Orthogonality Verification
      iii. Visualization of the SVD
      iv. Geometric Interpretation
      v. Visualization Implementation
   (d) **Proofreading/Formatting**
3. **Eric Filipkowski**
   (a) **Methodology**
      i. Matrix Normalization
   (b) **Conclusion**
      i. Conclusion
4. **Efosa Owie**
   (a) **Background**
      i. SVD for Image Compression
   (b) **Methodology**
      i. SVD Computation
5. **Isabel Cole**
   (a) **Introduction**
      i. Introduction
   (b) **Background**
      i. Singular Value Decomposition